

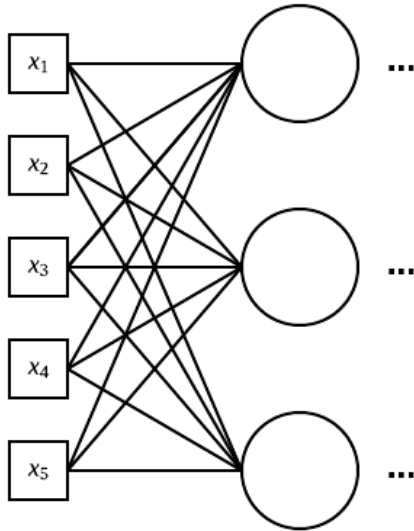


Deep learning (applications to segmentation)

Dr Cian M Scannell, Assistant Professor

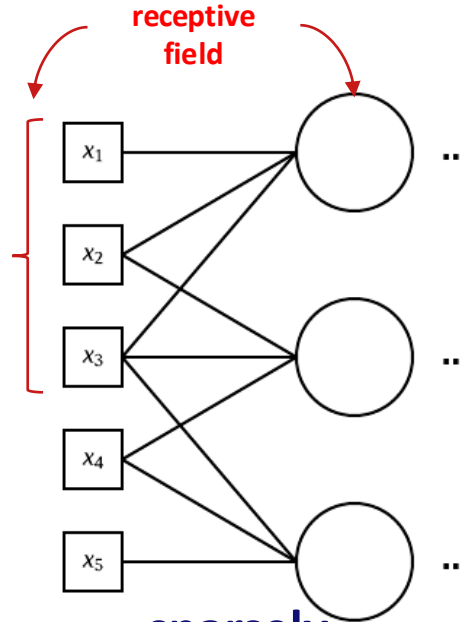
Department of Biomedical Engineering, Medical Image Analysis group

Previous lecture

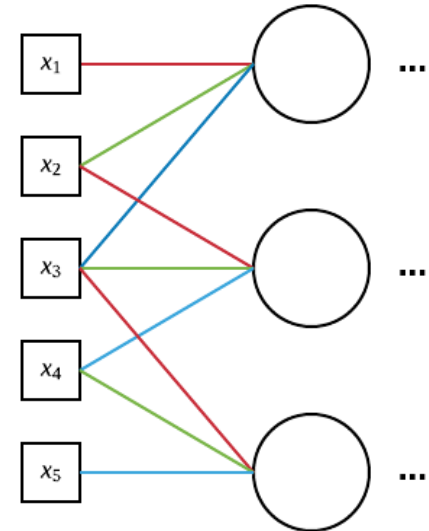


“regular” NN
15 weights

Reducing # of weights



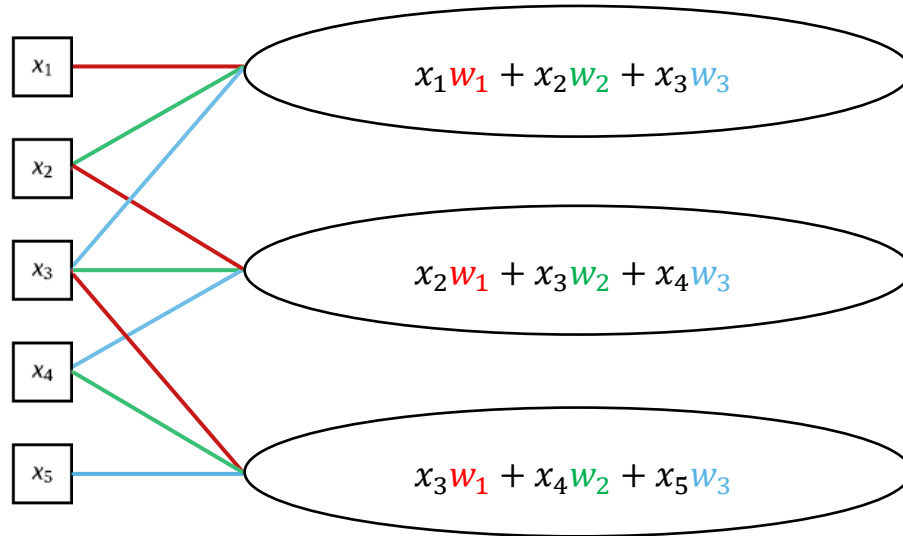
**sparsely
connected NN**
9 weights



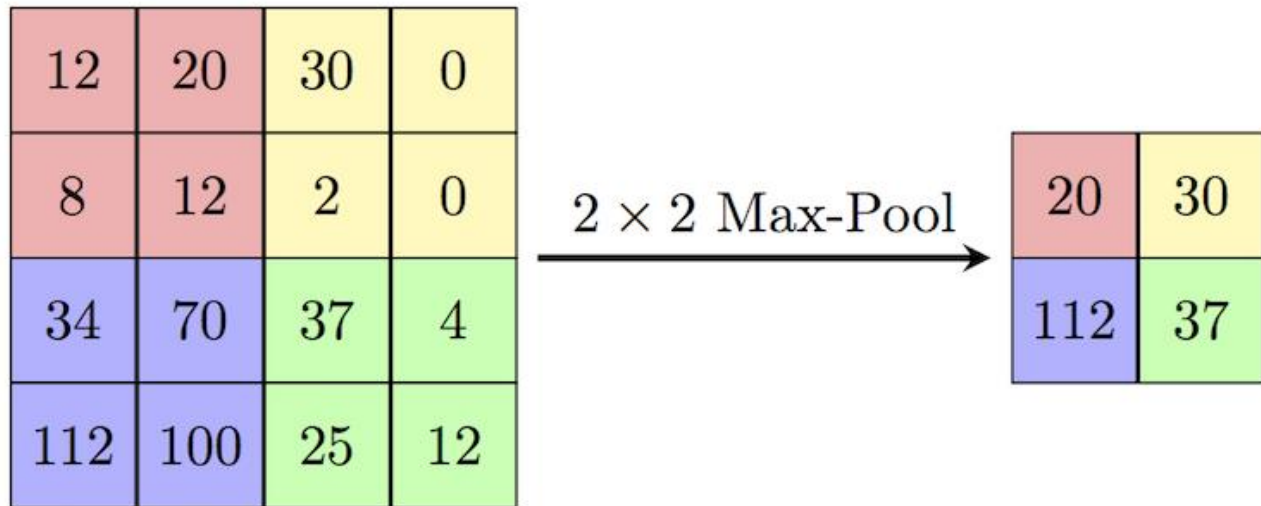
shared weights
3 weights

Previous lecture

1-D Convolution



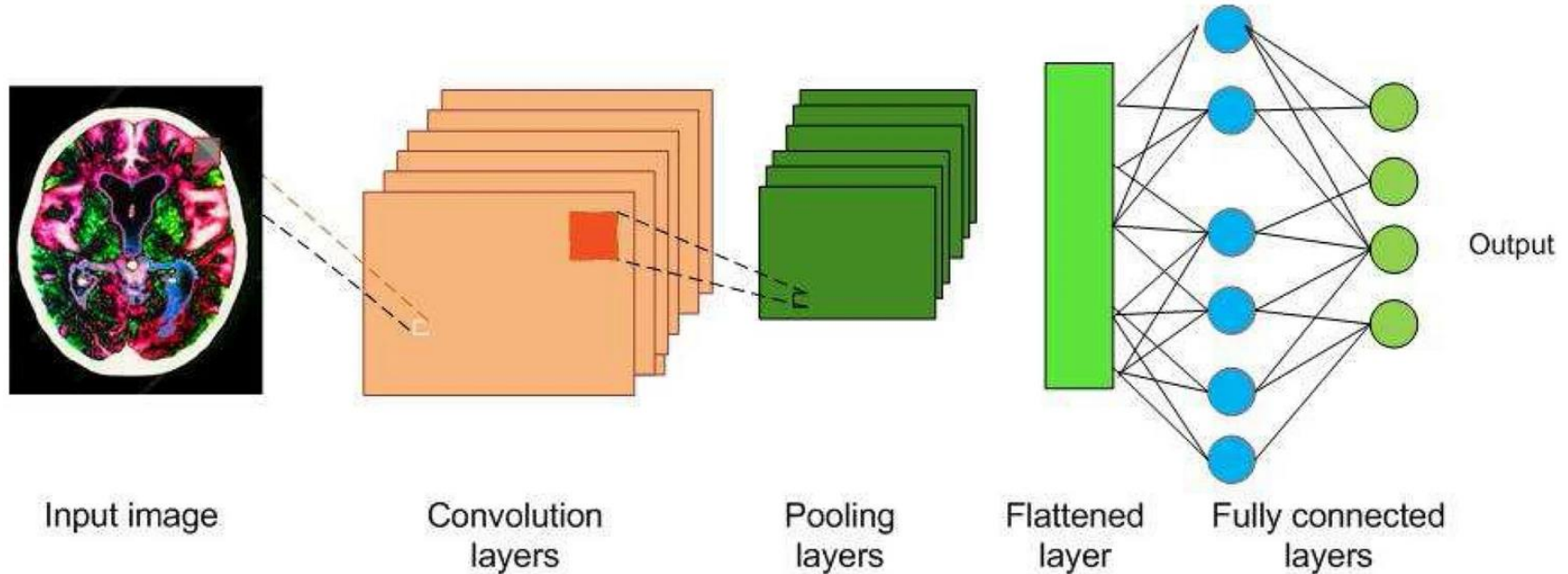
Previous lecture



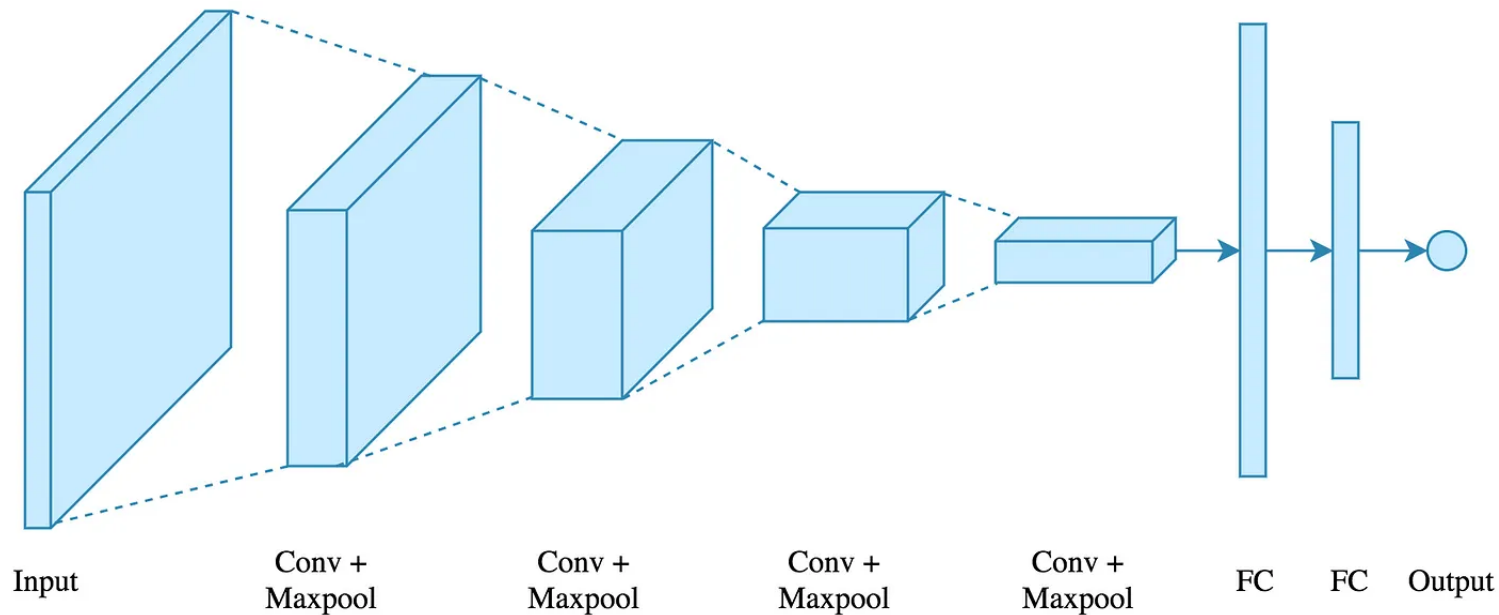
From layers to networks

Architectures

overall structure and organization of the network

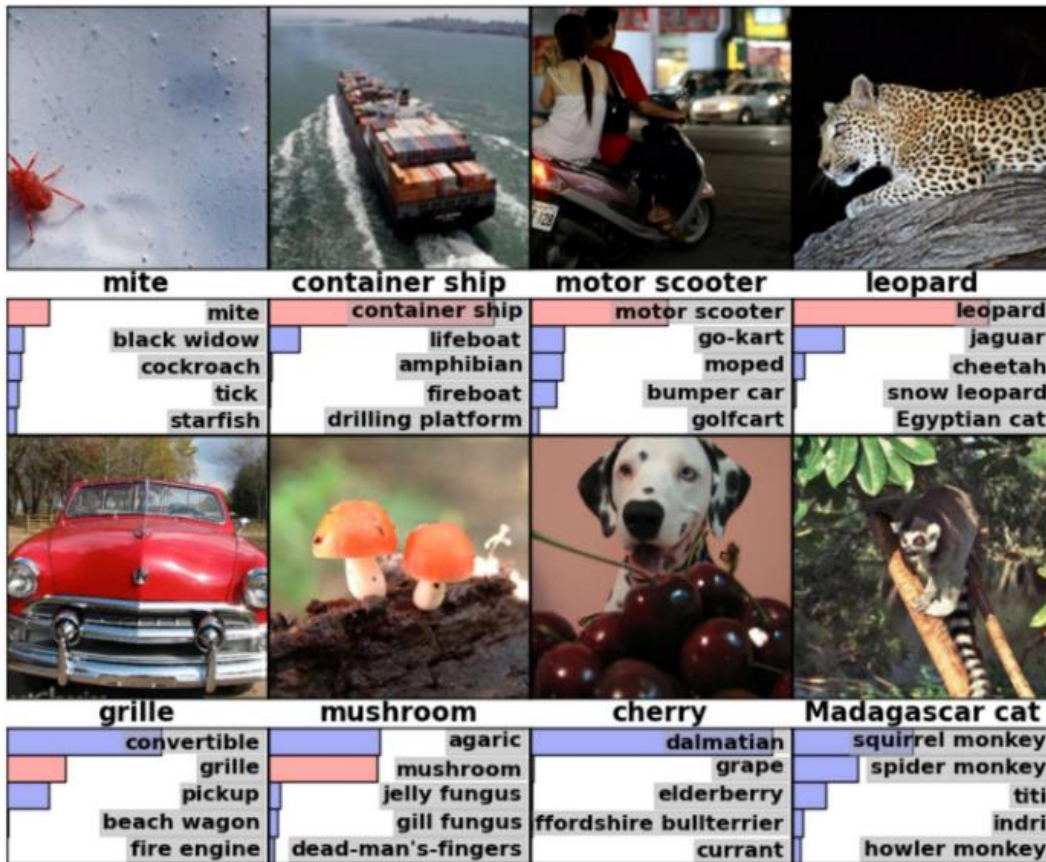


Architectures

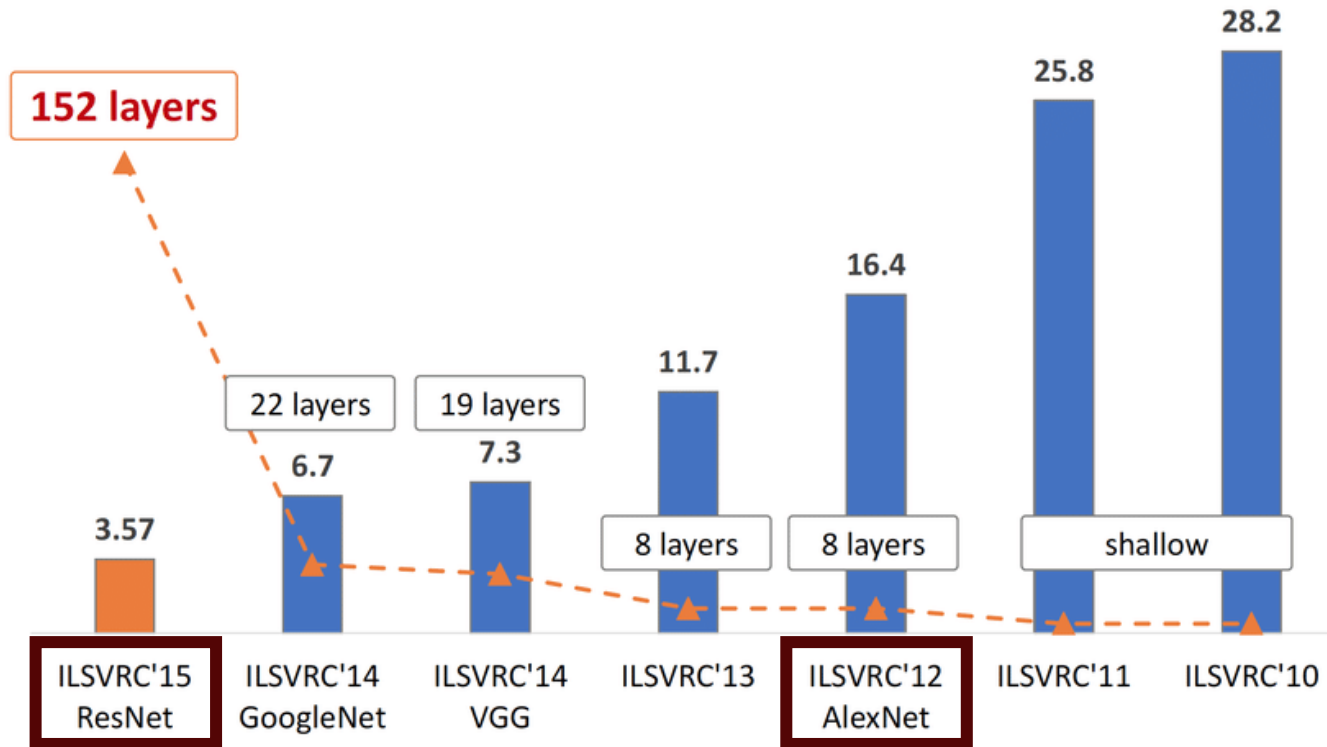


IMAGENET

- 1,000 object classes (categories).
- Images:
 - 1.2 M train
 - 100k test.

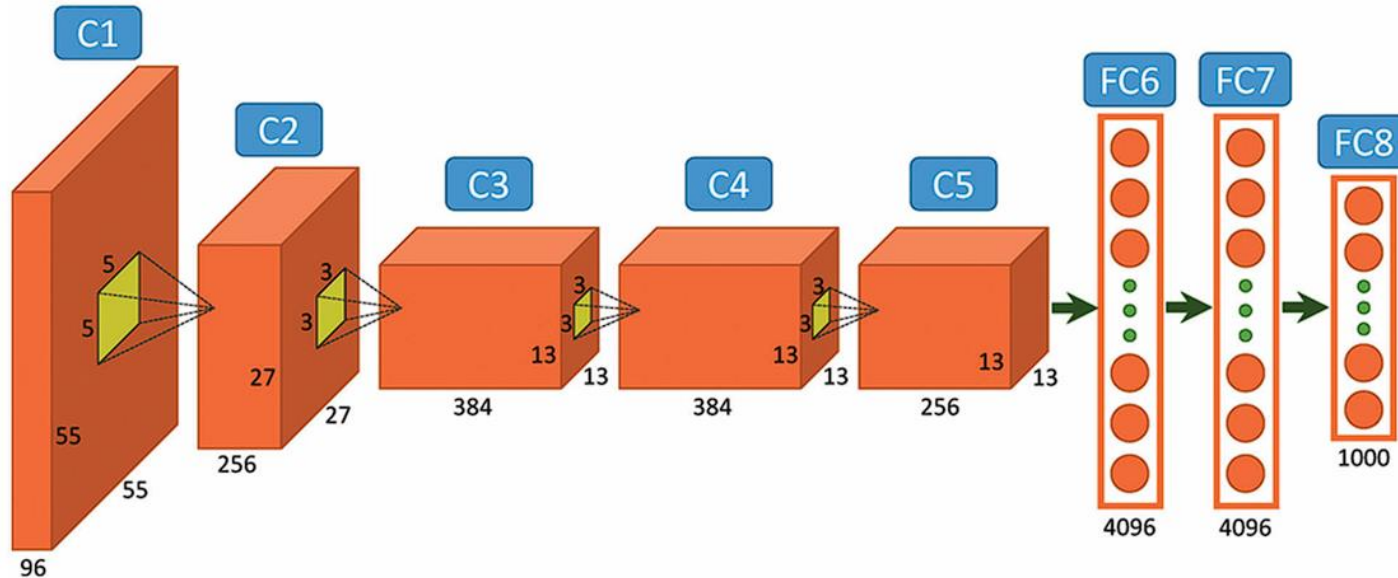


Error rate ImageNet – Benefits of scale



AlexNet

Architecture: It consisted of 5 convolutional layers followed by 3 fully connected layers, making it much deeper than previous networks.



Deeper networks

Neural networks have been around ...

**LEARNING INTERNAL REPRESENTATIONS
BY ERROR PROPAGATION**

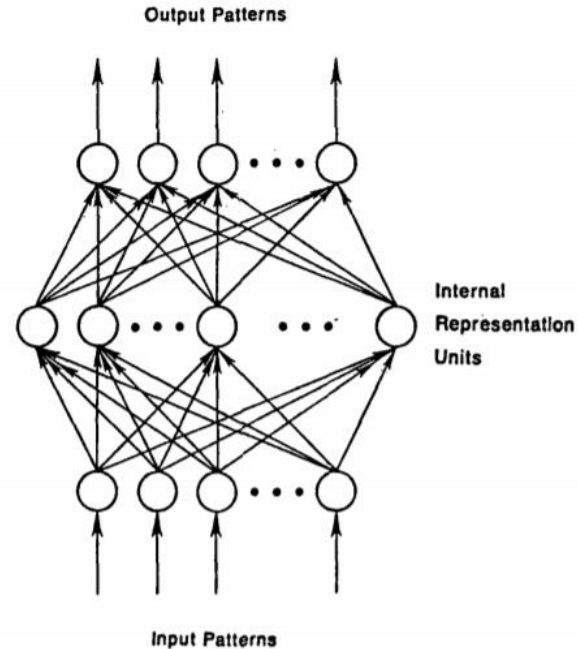
**David E. Rumelhart, Geoffrey E. Hinton,
and Ronald J. Williams**

September 1985

ICS Report 8506

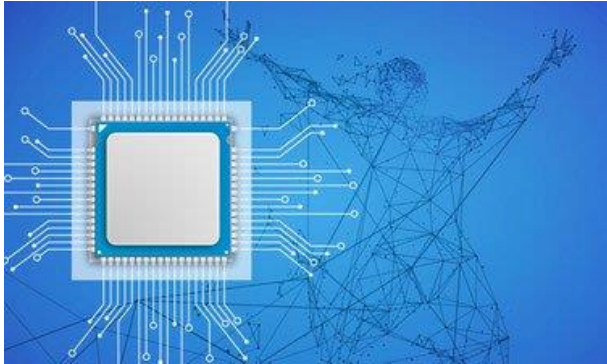


**COGNITIVE
SCIENCE**

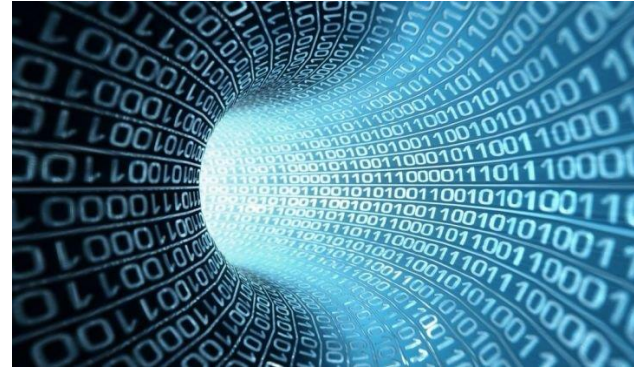


GPU

Why did it take so long to develop these deep learning models?

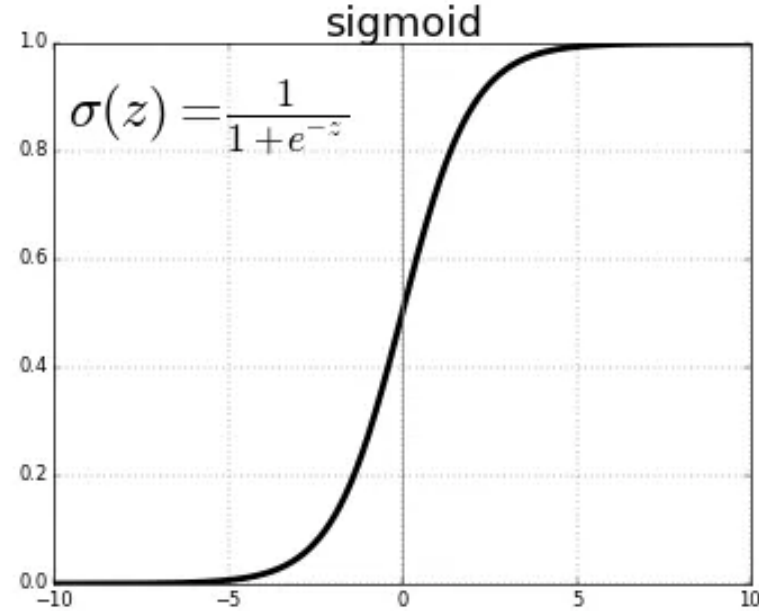
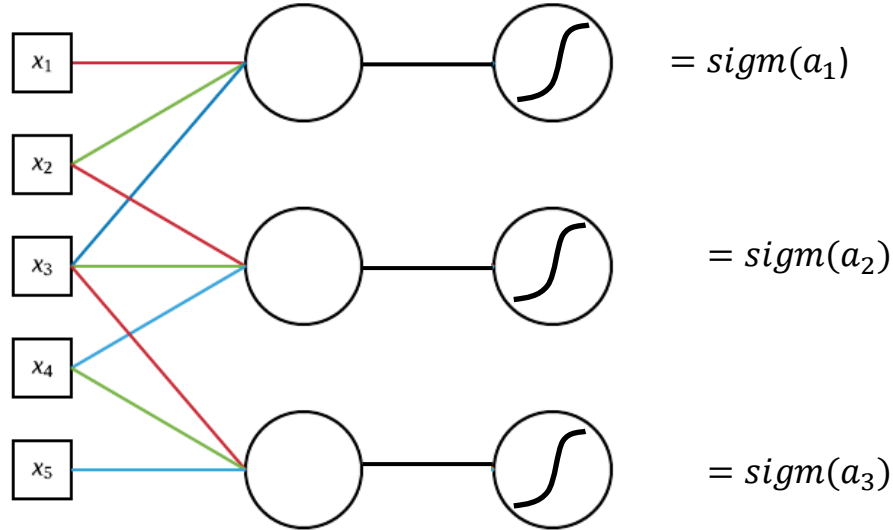


- Hardware improvements
- Parallel computing (GPU)

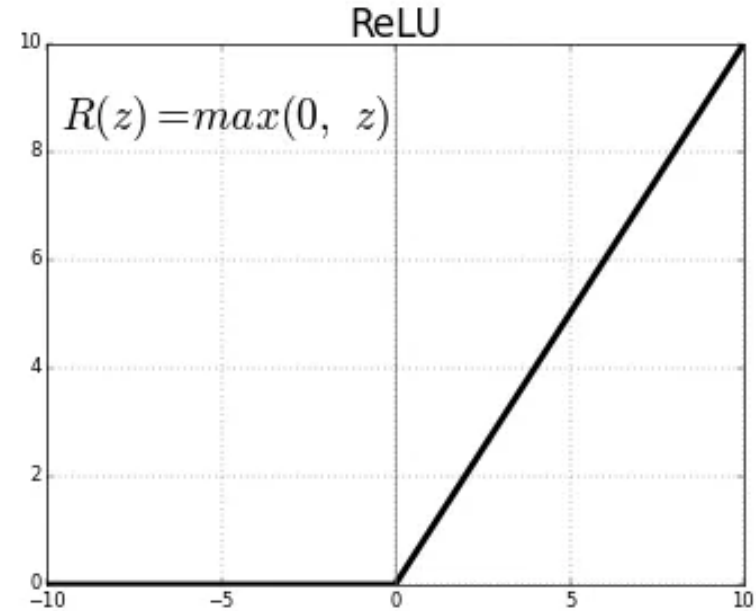
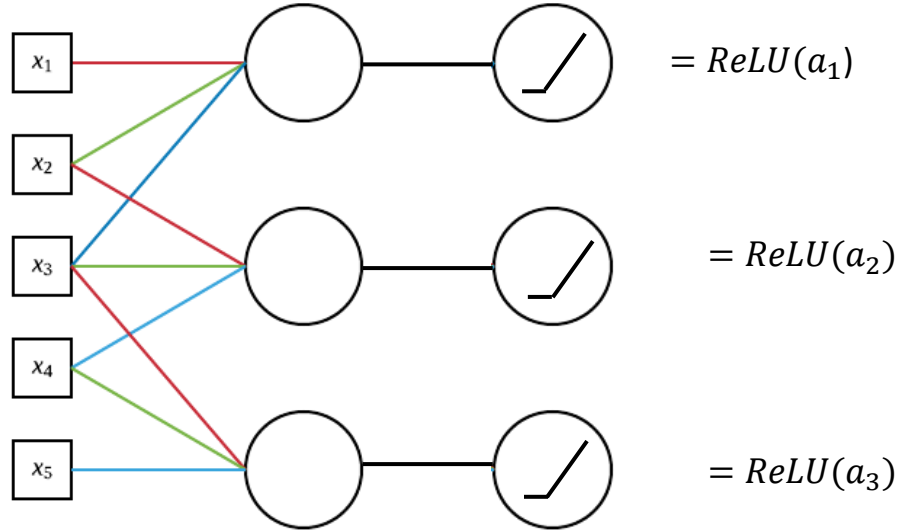


- Digital data
 - More (image) data
- = also true for medical imaging

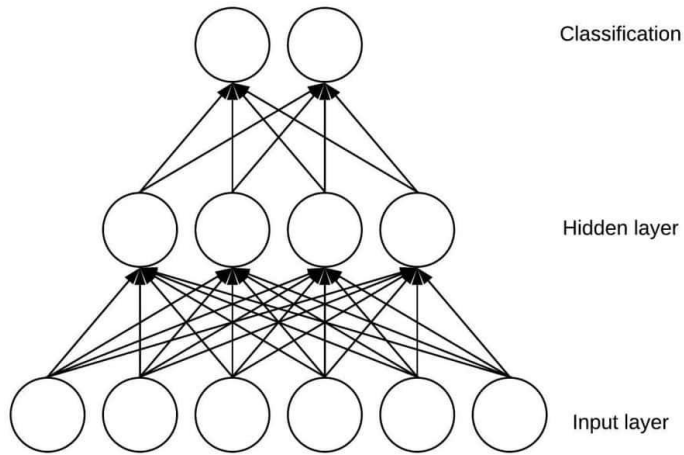
Convolutions + non-linearity (activation)



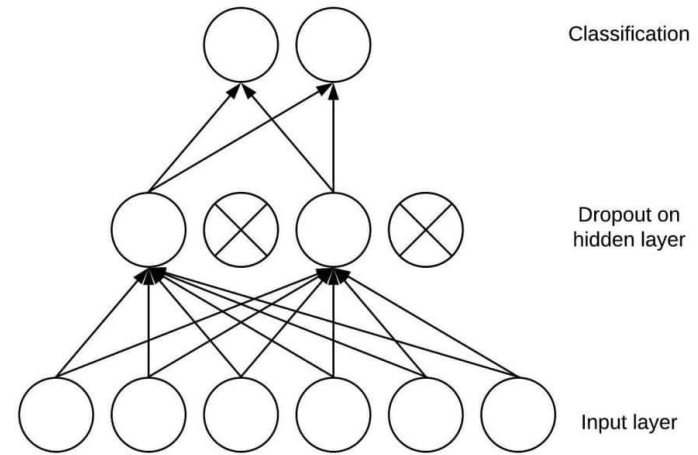
ReLU activation



Dropout

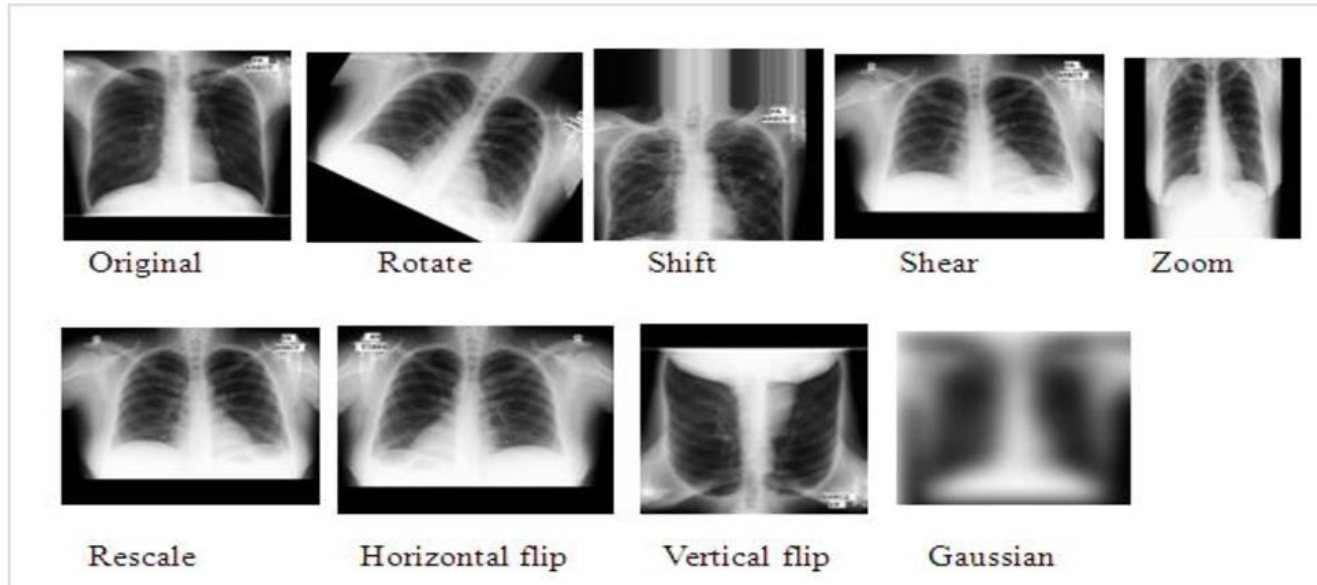


Without Dropout

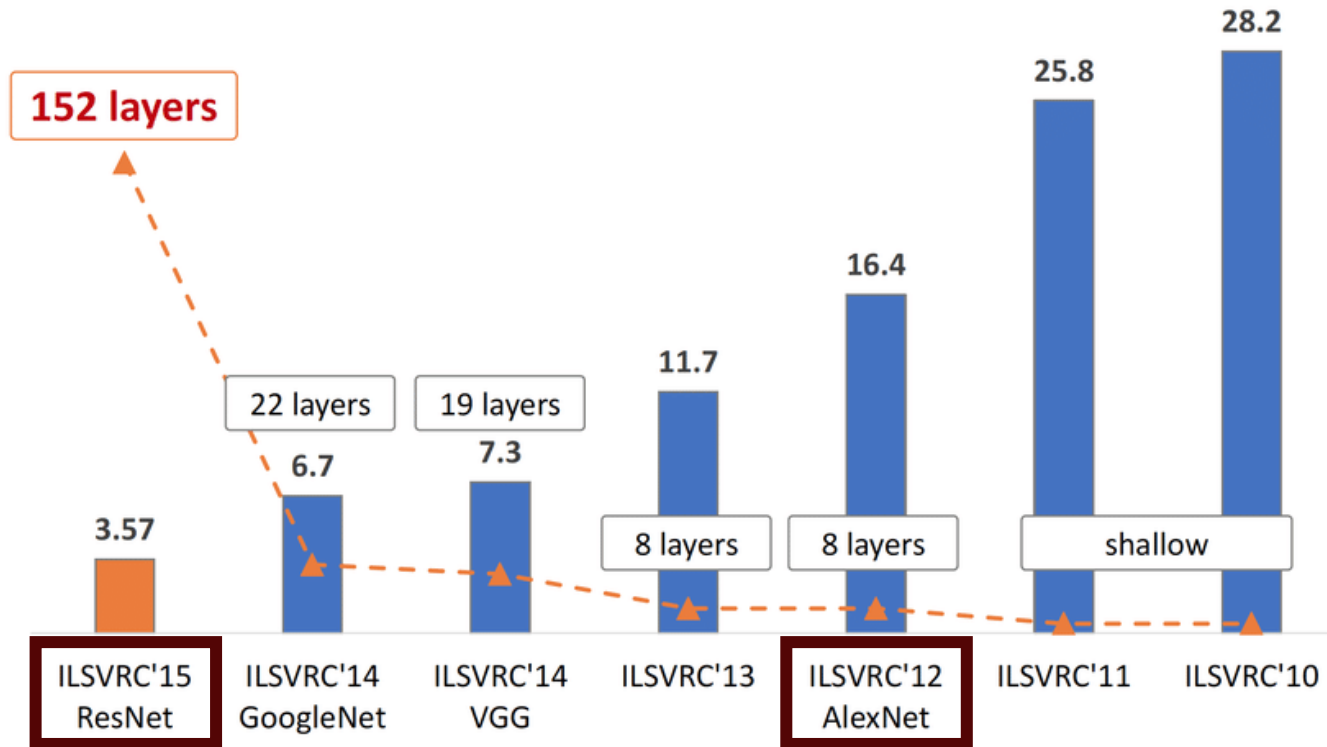


With Dropout

Augmentation



Error rate ImageNet – Benefits of scale



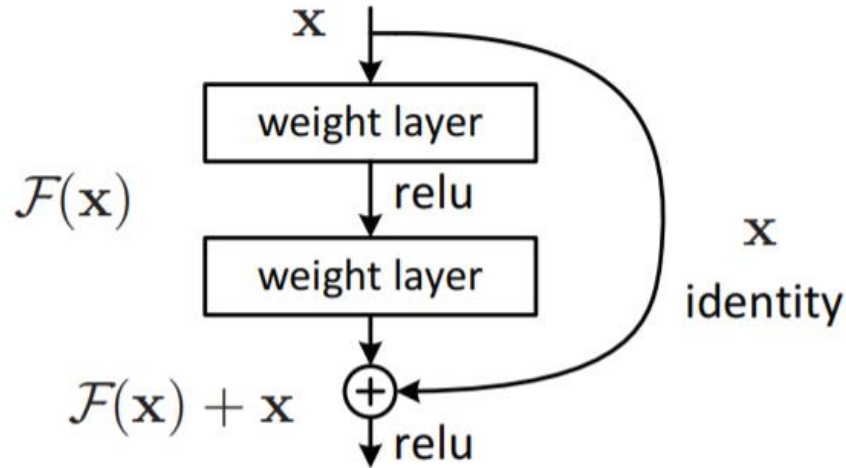
ResNet (residual network)

34-layer plain



Residual (skip) connections

Learn differences between input and output layer = residue



$H(x) = F(x) + x$ where:

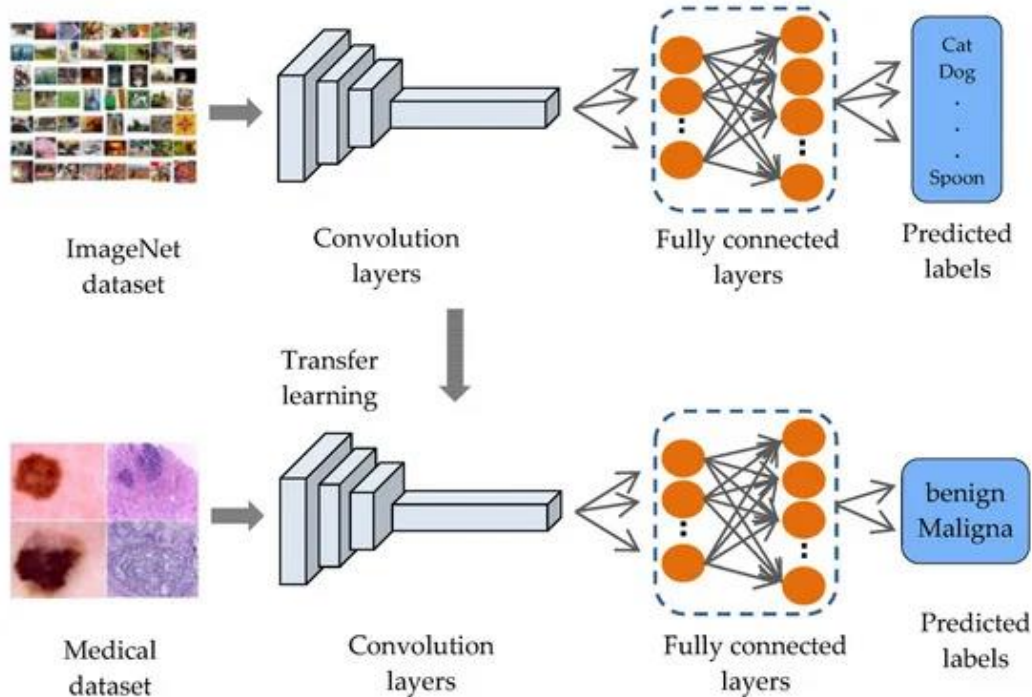
$H(x)$ is the desired underlying mapping

x is the input

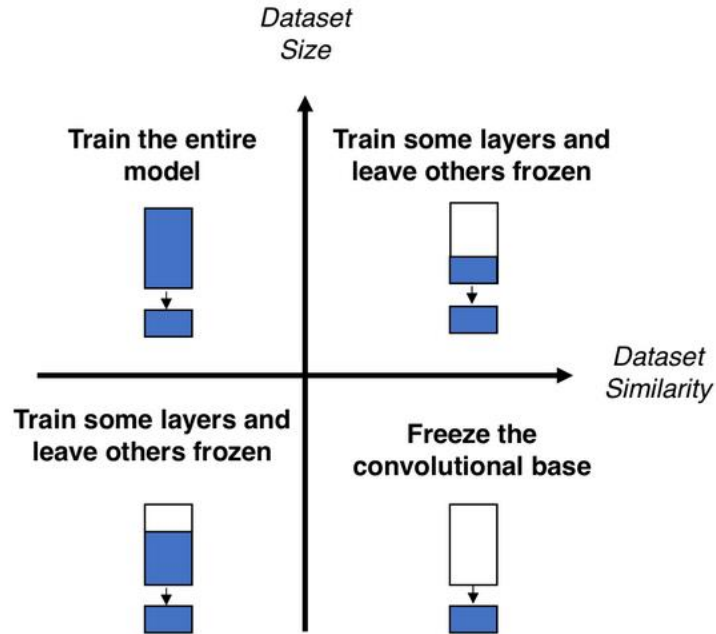
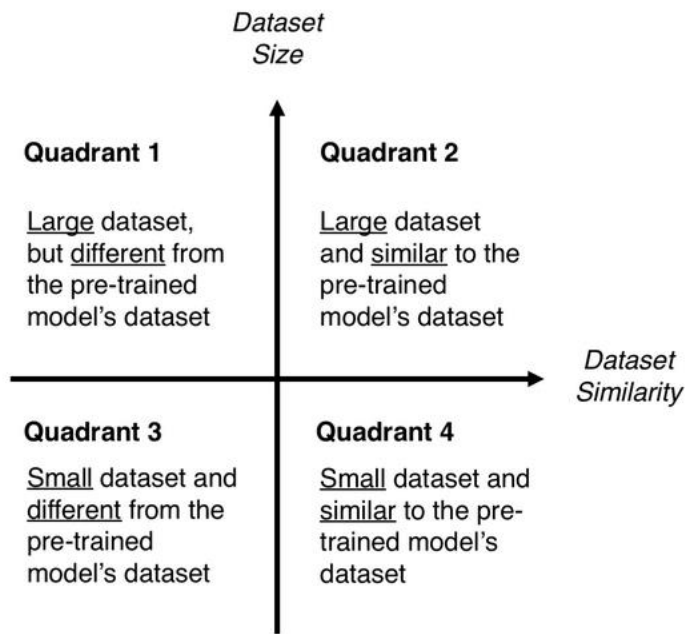
Benefits of residual connections

- Mitigates vanishing gradients
 - direct path for gradients to flow backward
- Makes optimisation easier
 - easier to optimize residual mappings than to learn an entire transformation
- Information preservation
 - shortcut preserves important features learned in earlier layers

ImageNet (continued) – Transfer learning

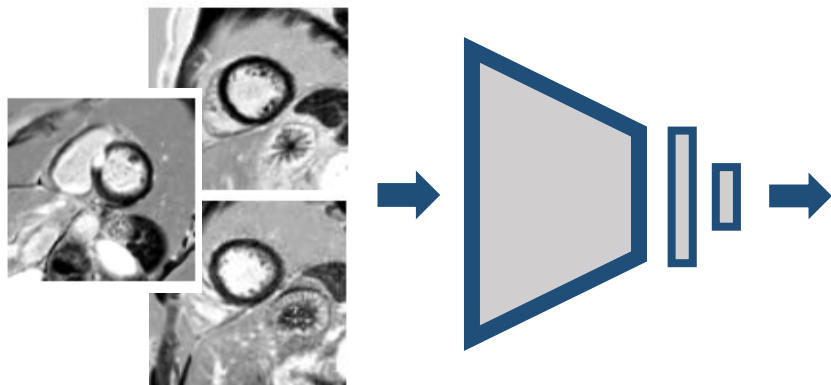


Transfer learning

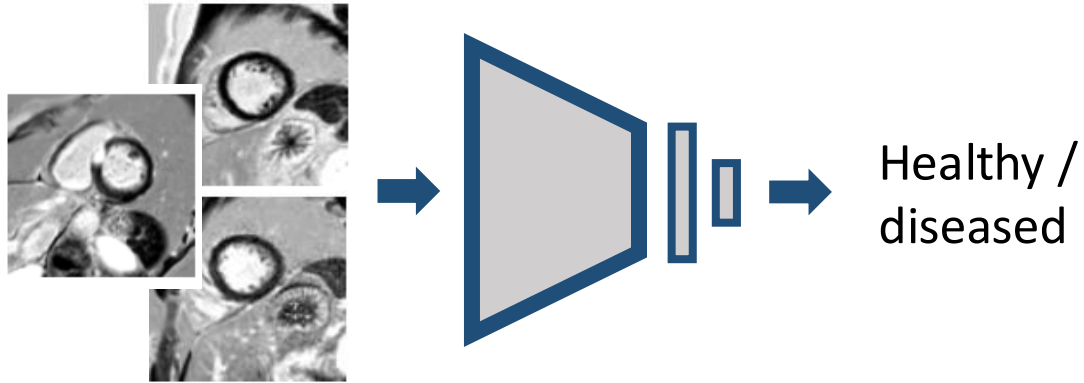


Deep learning in medical image analysis

CNNs



Classification



Classification

Two classes

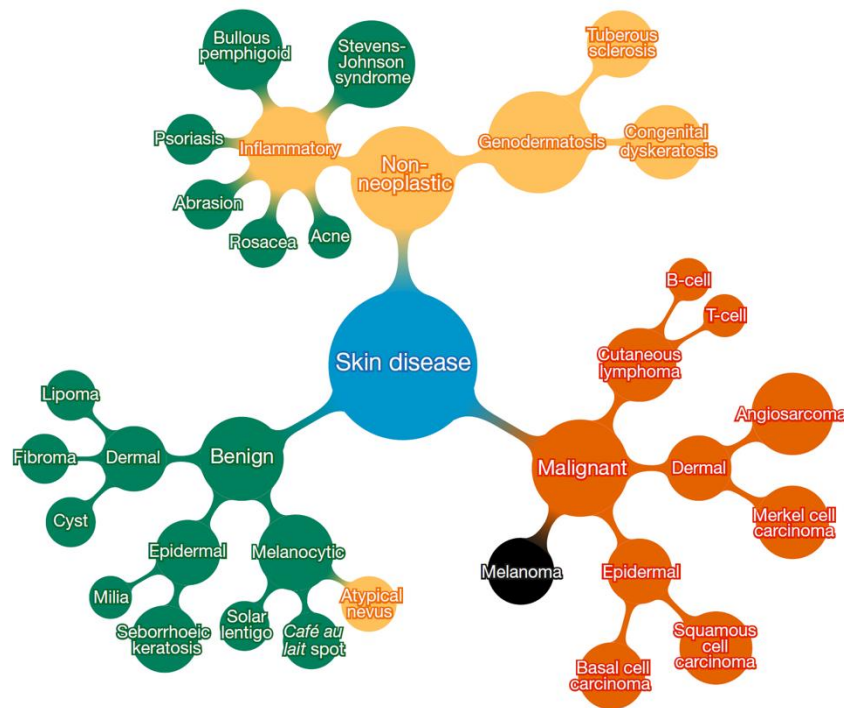
e.g. malignant lesions / other

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

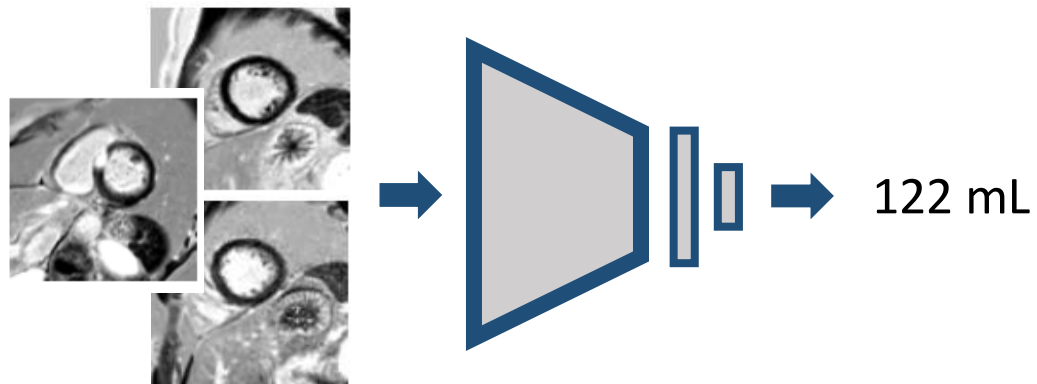
Multi-class

benign lesions / malignant lesions /
non-neoplastic

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$



Regression



Regression

Linear output layer

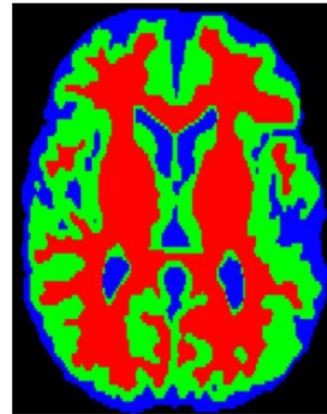
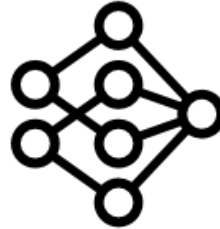
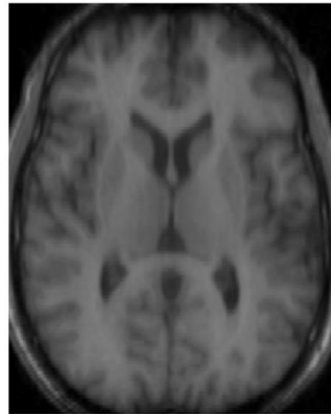
$$\sigma(z) = z$$

Deep learning for image segmentation

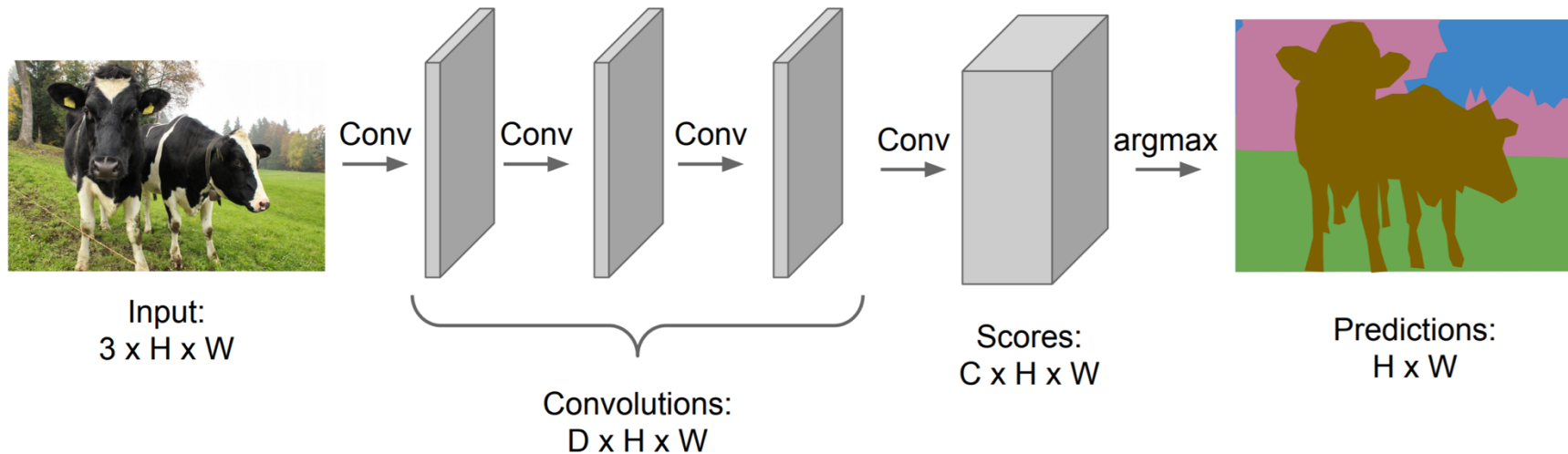
Segmentation?

The limitation of the architectures considered so far is that they make a one (or a small few) output prediction(s) for an input image

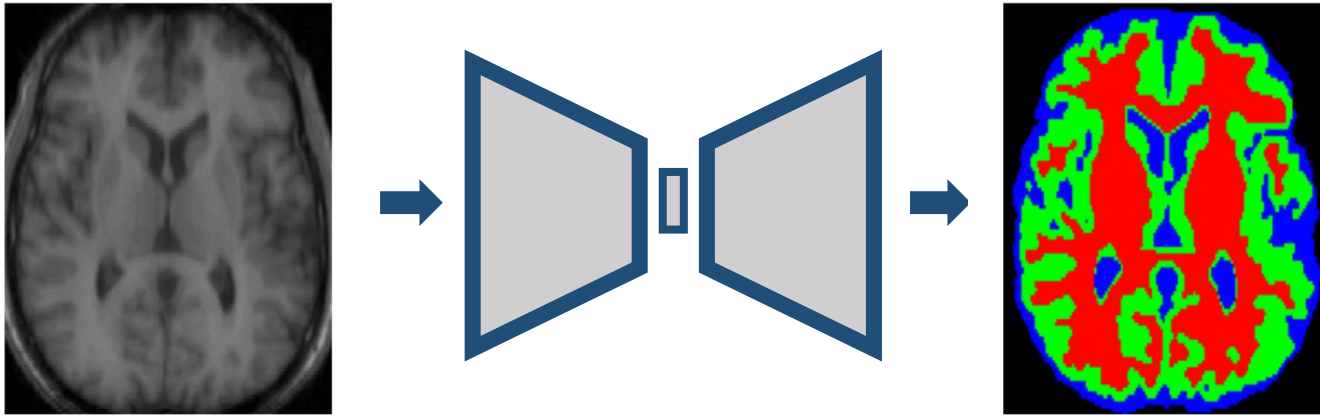
What if we want to make one prediction per pixel/voxel (e.g. pixelwise segmentation)?



Fully convolutional model

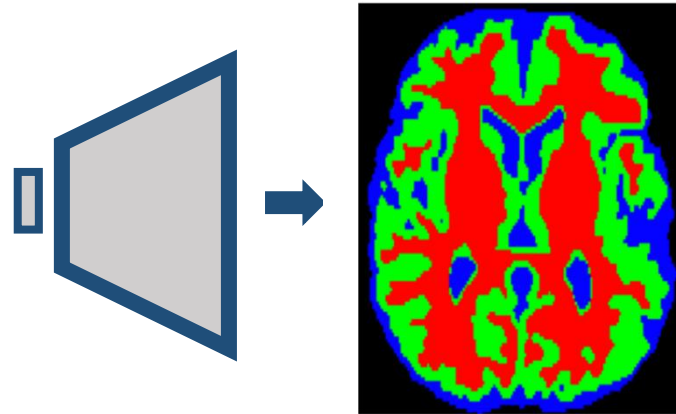


Encoder-decoder



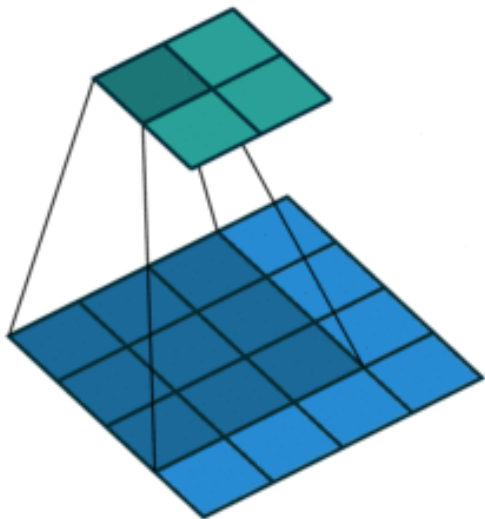
Decoder

Decoder needs to upsample low dimensional features

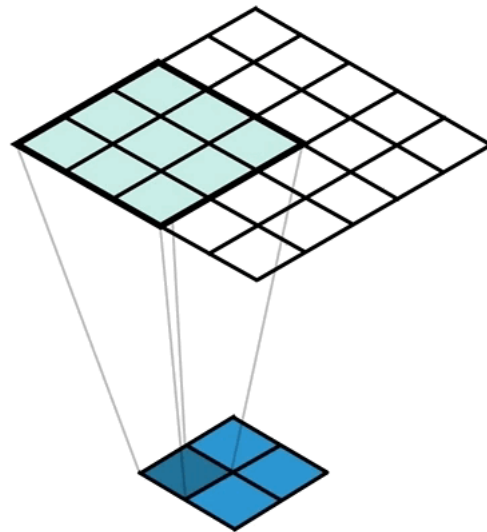


Transposed convolutions

From



To



Transposed Convolutions

2x2 convolution, stride of 1 and a pad of 0

$$\begin{aligned}4 * 3 &= 12 \\2 * 1 &= 2 \\12 + 2 &= 14\end{aligned}$$

6	14	4
2	17	21
0	1	5

Output

2	4
0	1

Input

3	1
1	5

Conv
Kernel

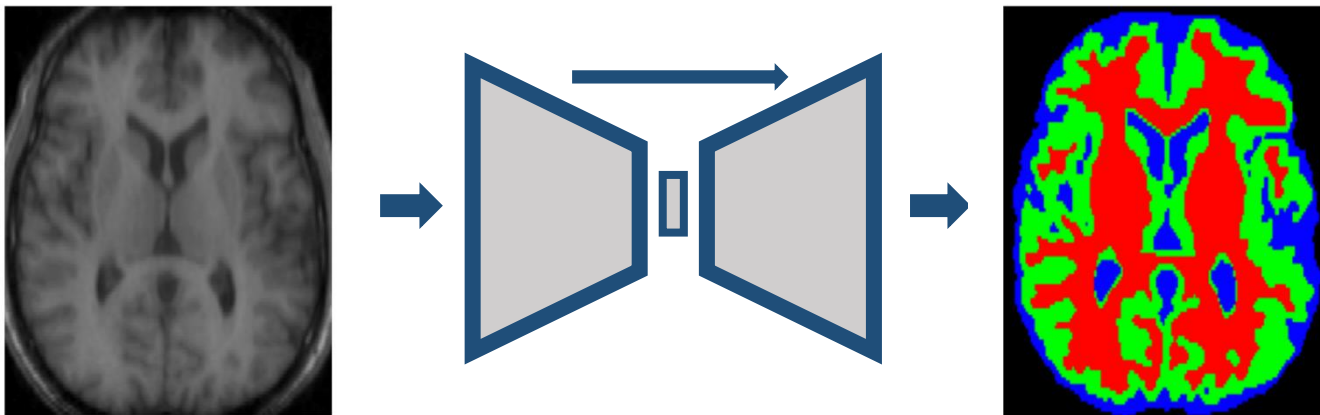
6	2	
2	10	

	12	4
	4	20

0	0	
0	0	

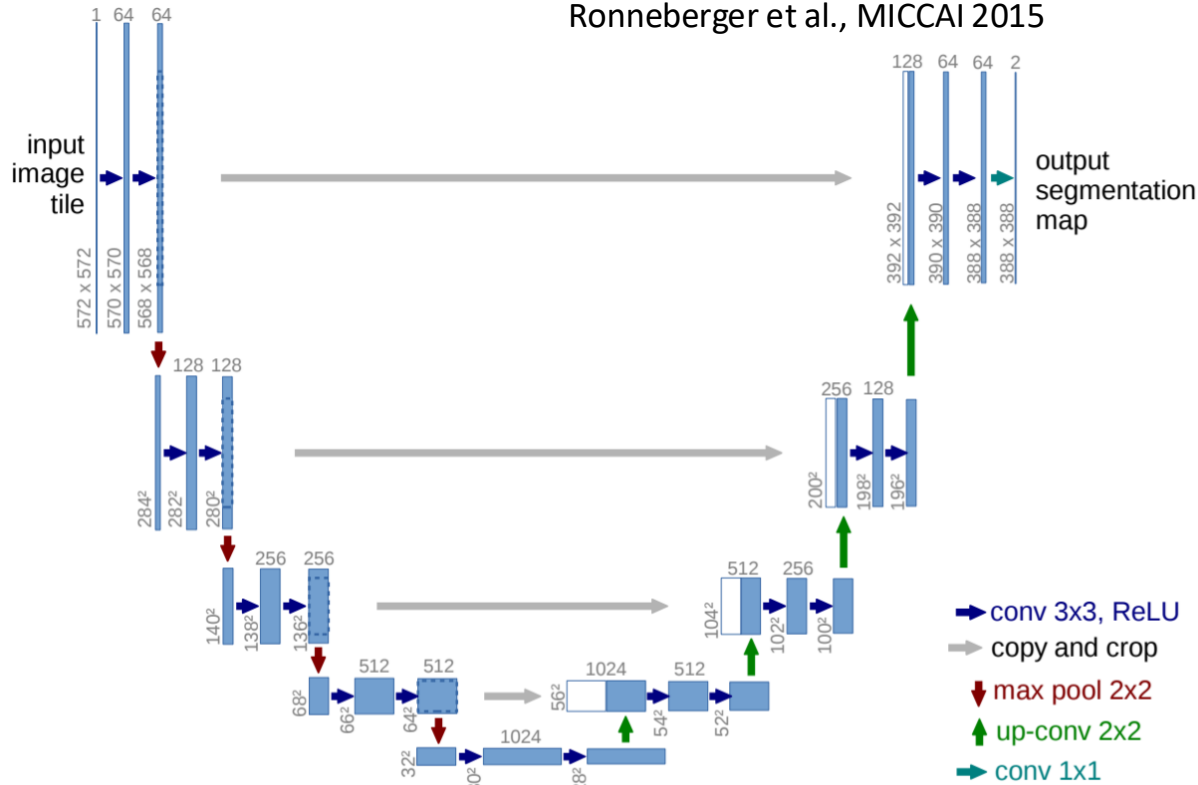
	3	1
	1	5

U-Net

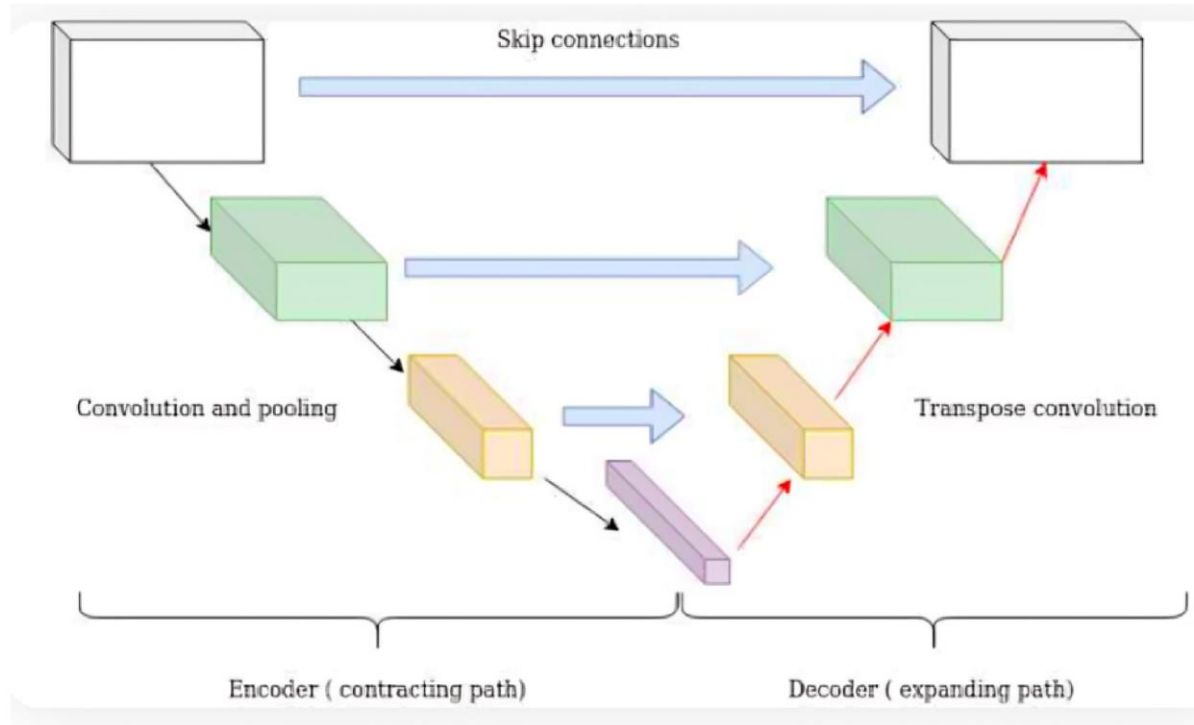


U-Net:

Ronneberger et al., MICCAI 2015



U-Net:



Encoder

Repeated application (usually 2) of 2D Convolution (3×3) with ReLU activation

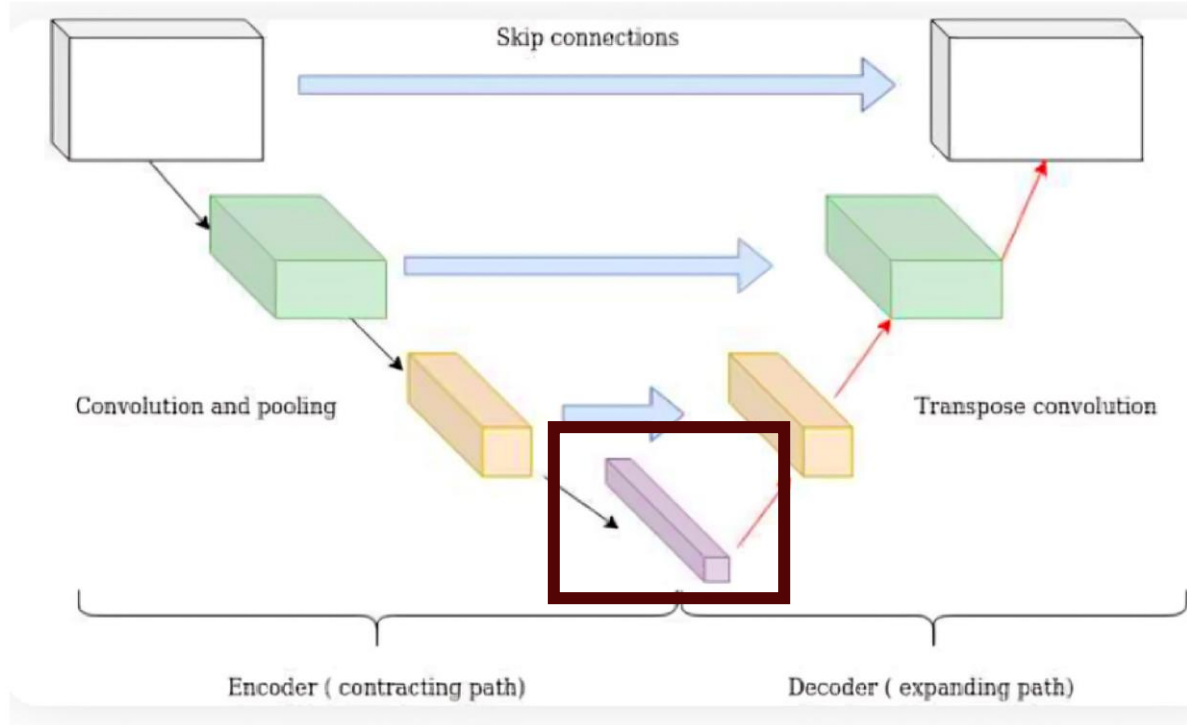
As the image passes through successive layers, the encoder creates a hierarchical representation:

- Early layers capture low-level features (e.g., edges and corners).
- Deeper layers capture high-level semantic features (e.g., shapes and object parts).

Encoder

- By progressively reducing spatial dimensions, the encoder focuses on capturing essential information while ignoring irrelevant details.
- Outputs from different stages of the encoder provide features at multiple resolutions, enabling the network to analyze both fine and coarse details

Bottleneck



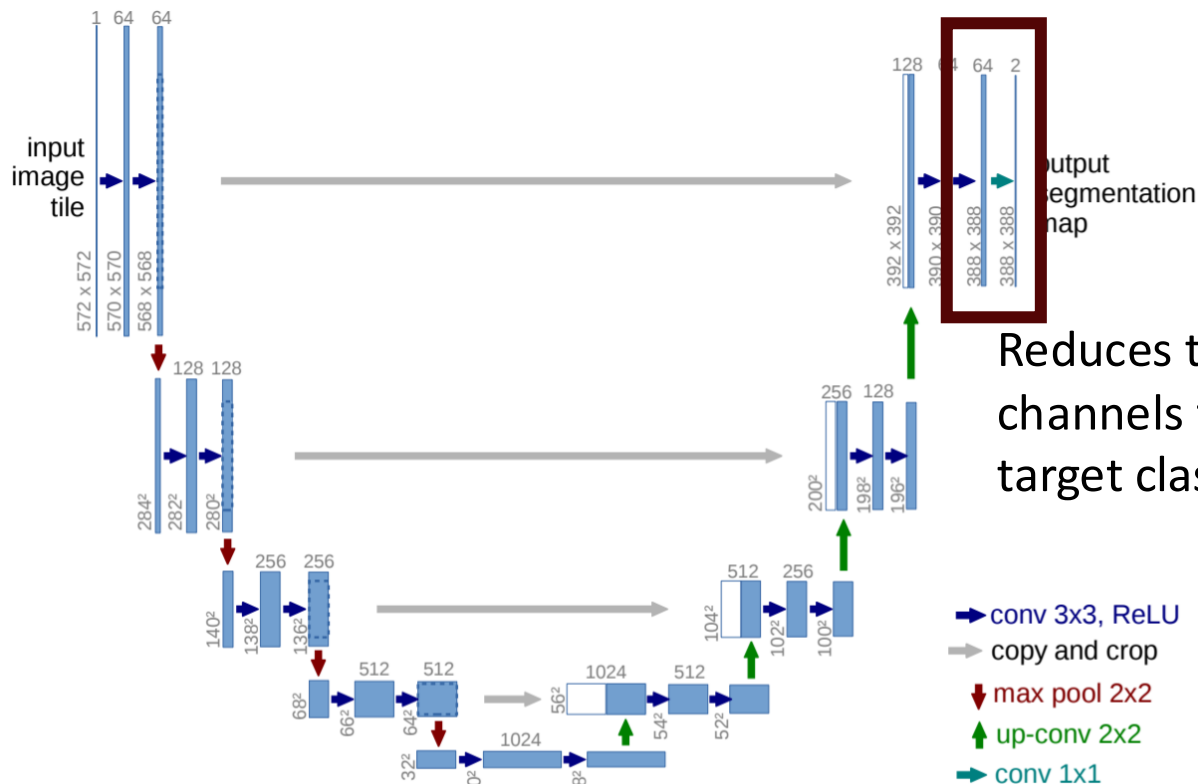
This is the "bridge" between the encoder and decoder. Contains convolutional layers with no downsampling.

Decoder

Restores spatial resolution while combining semantic information from deeper layers with spatial details from earlier layers.

- Upsample
- Concatenate with features from skip connections
- Convolutional layers (with activations)

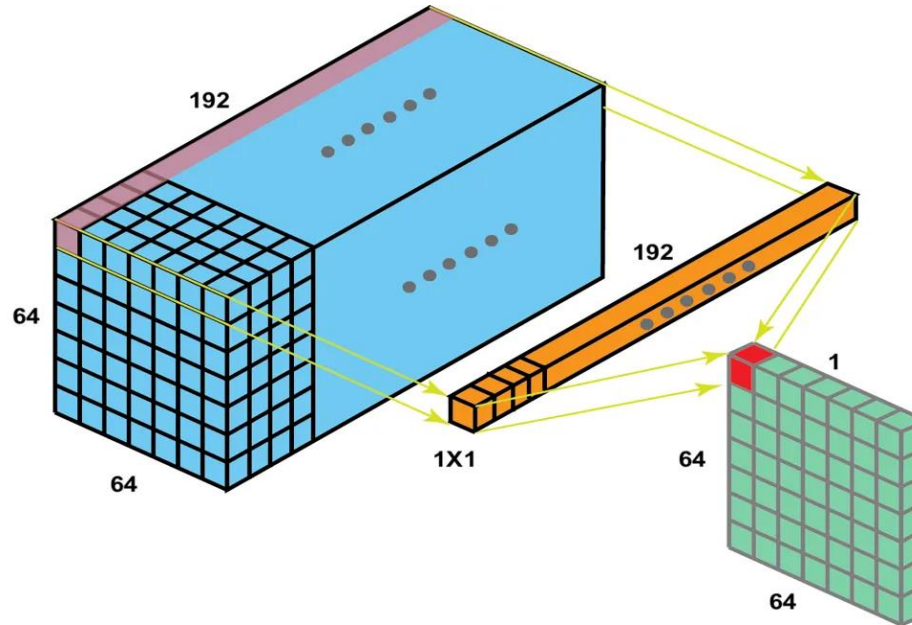
1x1 convolution



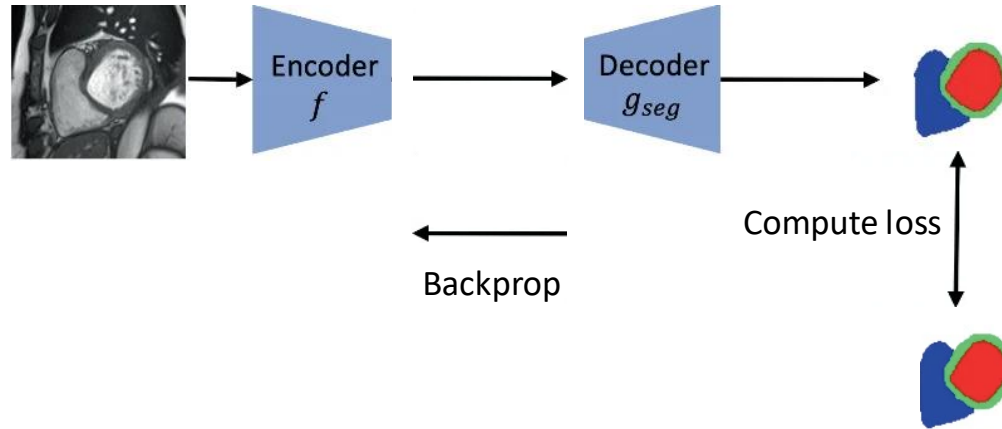
Reduces the feature channels to the number of target classes

1x1 convolution

The 1x1(x192) convolution applies a linear transformation based on the input channels.

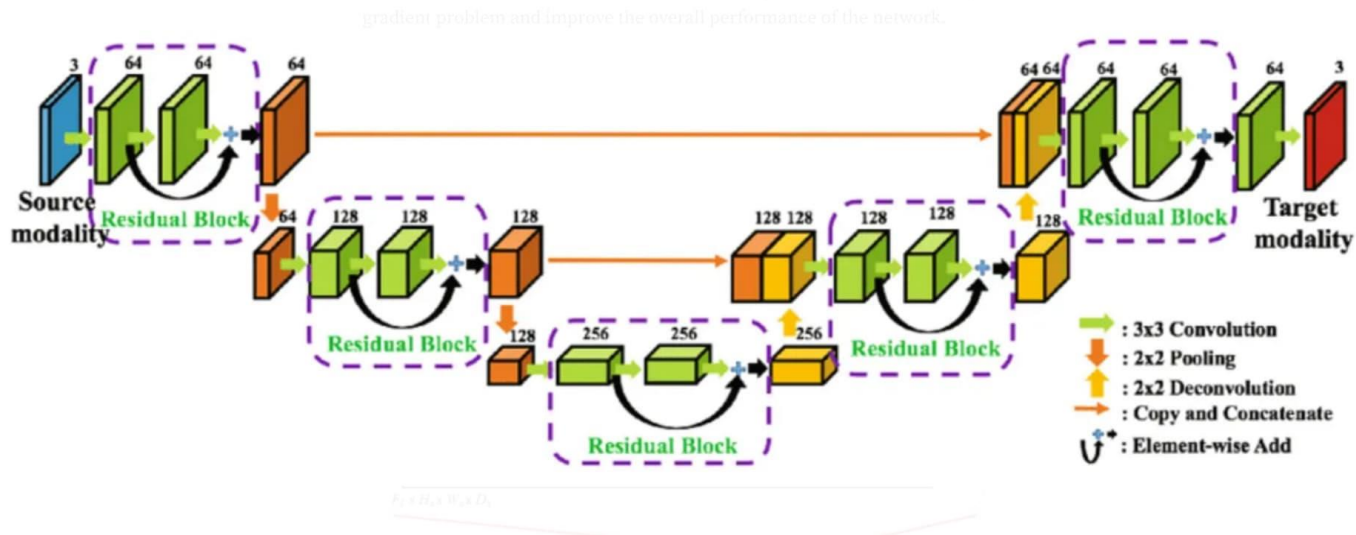


Training



E.g. treat as a pixelwise classification problem and use sum of pixelwise classification loss (binary/multiclass cross-entropy) for training

Residual U-Net



A (pretrained) ResNet can be used for the encoder/down sampling section of the U-Net (the left half of the U).

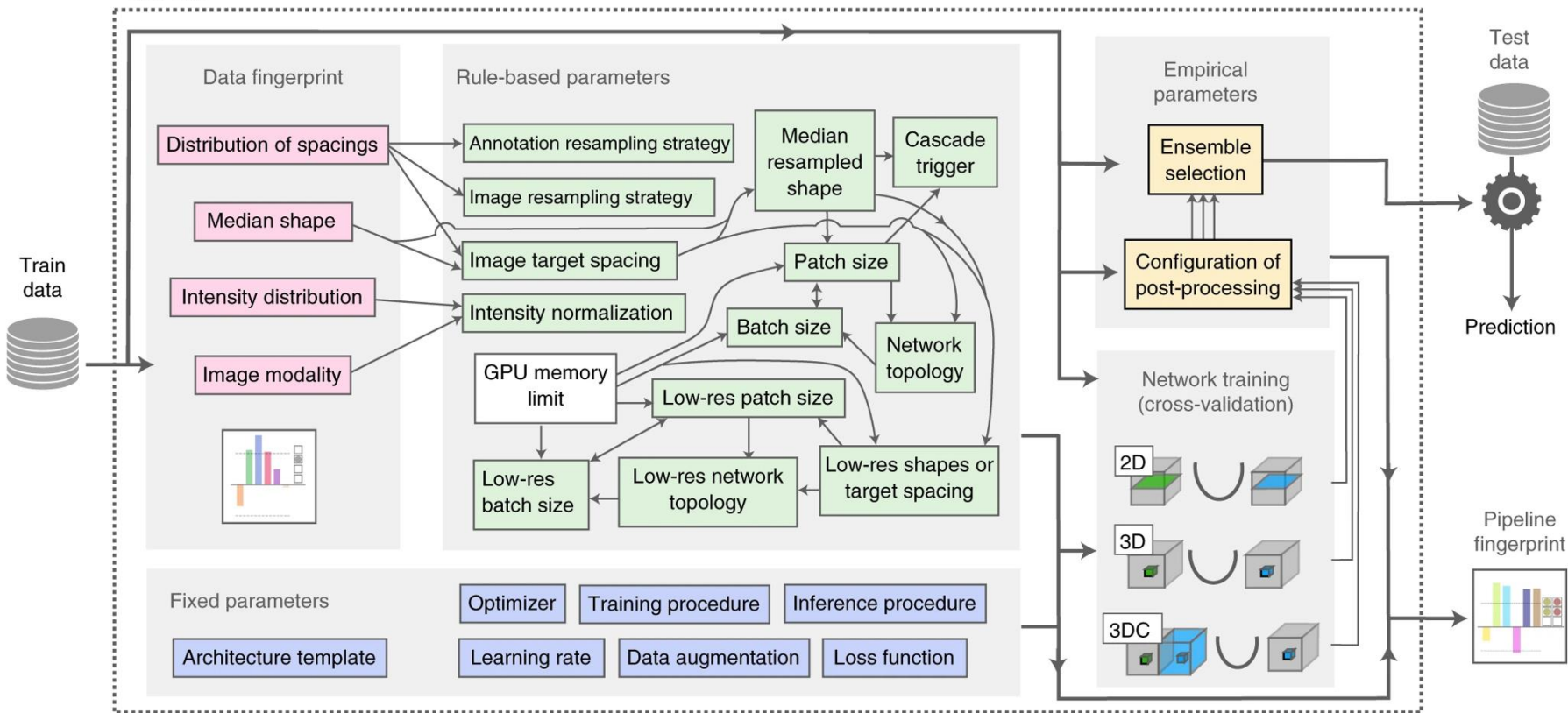
But not all U-Nets are the same

Segmentation is still challenging due to the vast number of different imaging modalities and anatomies, each of perform best with a different preprocessing and model approach

Broadly similar methods can give very different results due to implementation

nnU-Net 'no new net'

nnU-net



Design choice	Required input	Automated (fixed, rule-based or empirical) configuration derived by distilling expert knowledge (more details in online methods)
Learning rate	–	Poly learning rate schedule (initial, 0.01)
Loss function	–	Dice and cross-entropy
Architecture template	–	Encoder–decoder with skip-connection ('U-Net-like') and instance normalization, leaky ReLU, deep supervision (topology-adapted in inferred parameters)
Optimizer	–	SGD with Nesterov momentum ($\mu = 0.99$)
Data augmentation	–	Rotations, scaling, Gaussian noise, Gaussian blur, brightness, contrast, simulation of low resolution, gamma correction and mirroring
Training procedure	–	1,000 epochs $\times$ 250 minibatches, foreground oversampling
Inference procedure	–	Sliding window with half-patch size overlap, Gaussian patch center weighting
Intensity normalization	Modality, intensity distribution	If CT, global dataset percentile clipping & z score with global foreground mean and s.d. Otherwise, z score with per image mean and s.d.
Image resampling strategy	Distribution of spacings	If anisotropic, in-plane with third-order spline, out-of-plane with nearest neighbor Otherwise, third-order spline
Annotation resampling strategy	Distribution of spacings	Convert to one-hot encoding $\rightarrow$ If anisotropic, in-plane with linear interpolation, out-of-plane with nearest neighbor Otherwise, linear interpolation

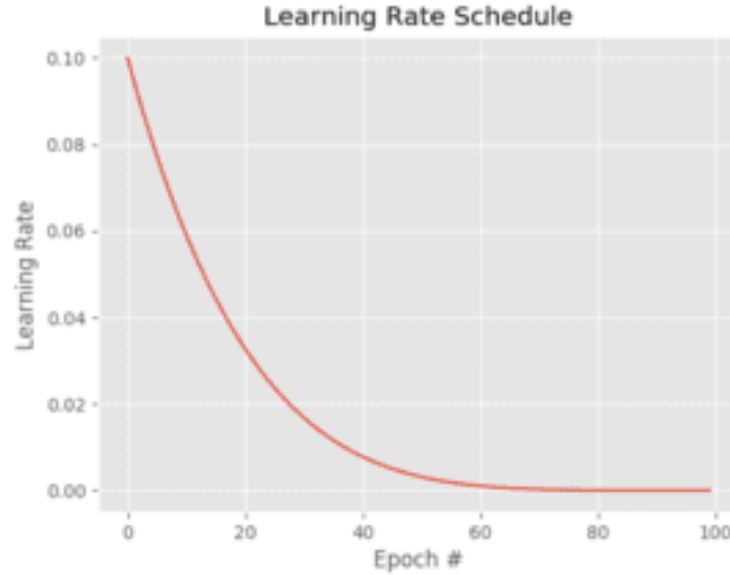
Image target spacing	Distribution of spacings	If anisotropic, lowest resolution axis tenth percentile, other axes median. Otherwise, median spacing for each axis. (computed based on spacings found in training cases)
Network topology, patch size, batch size	Median resampled shape, target spacing, GPU memory limit	Initialize the patch size to median image shape and iteratively reduce it while adapting the network topology accordingly until the network can be trained with a batch size of at least 2 given GPU memory constraints. for details see online methods.
Trigger of 3D U-Net cascade	Median resampled image size, patch size	Yes, if patch size of the 3D full resolution U-Net covers less than 12.5% of the median resampled image shape
Configuration of low-resolution 3D U-Net	Low-res target spacing or image shapes, GPU memory limit	Iteratively increase target spacing while reconfiguring patch size, network topology and batch size (as described above) until the configured patch size covers 25% of the median image shape. For details, see online methods.
Configuration of post-processing	Full set of training data and annotations	Treating all foreground classes as one; does all-but-largest-component-suppression increase cross-validation performance? Yes, apply; reiterate for individual classes No, do not apply; reiterate for individual foreground classes
Ensemble selection	Full set of training data and annotations	From 2D U-Net, 3D U-Net or 3D cascade, choose the best model (or combination of two) according to cross-validation performance

nnU-Net “no new net”



Learning rate	–	Poly learning rate schedule (initial, 0.01)
Loss function	–	Dice and cross-entropy
Architecture template	–	Encoder–decoder with skip-connection ('U-Net-like') and instance normalization, leaky ReLU, deep supervision (topology-adapted in inferred parameters)
Optimizer	–	SGD with Nesterov momentum ($\mu = 0.99$)
Data augmentation	–	Rotations, scaling, Gaussian noise, Gaussian blur, brightness, contrast, simulation of low resolution, gamma correction and mirroring
Training procedure	–	1,000 epochs $\times$ 250 minibatches, foreground oversampling

nnU-Net

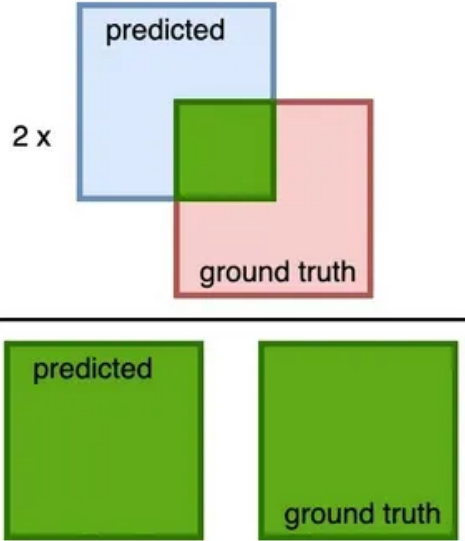


nnU-Net “no new net”



Learning rate	–	Poly learning rate schedule (initial, 0.01)
Loss function	–	Dice plus cross-entropy
Architecture template	–	Encoder–decoder with skip-connection ('U-Net-like') and instance normalization, leaky ReLU, deep supervision (topology-adapted in inferred parameters)
Optimizer	–	SGD with Nesterov momentum ($\mu = 0.99$)
Data augmentation	–	Rotations, scaling, Gaussian noise, Gaussian blur, brightness, contrast, simulation of low resolution, gamma correction and mirroring
Training procedure	–	1,000 epochs $\times$ 250 minibatches, foreground oversampling

Loss function

$$\text{Dice coefficient} = \frac{2 \times \text{area of overlapped (green)}}{\text{total area (green)}} =$$


Dice loss = 1-Dice coefficient

nnU-Net “no new net”



Learning rate	–	Poly learning rate schedule (initial, 0.01)
Loss function	–	Dice and cross-entropy
Architecture template	–	Encoder–decoder with skip-connection ('U-Net-like') and instance normalization, leaky ReLU, deep supervision (topology-adapted in inferred parameters)
Optimizer	–	SGD with Nesterov momentum ($\mu = 0.99$)
Data augmentation	–	Rotations, scaling, Gaussian noise, Gaussian blur, brightness, contrast, simulation of low resolution, gamma correction and mirroring
Training procedure	–	1,000 epochs $\times$ 250 minibatches, foreground oversampling

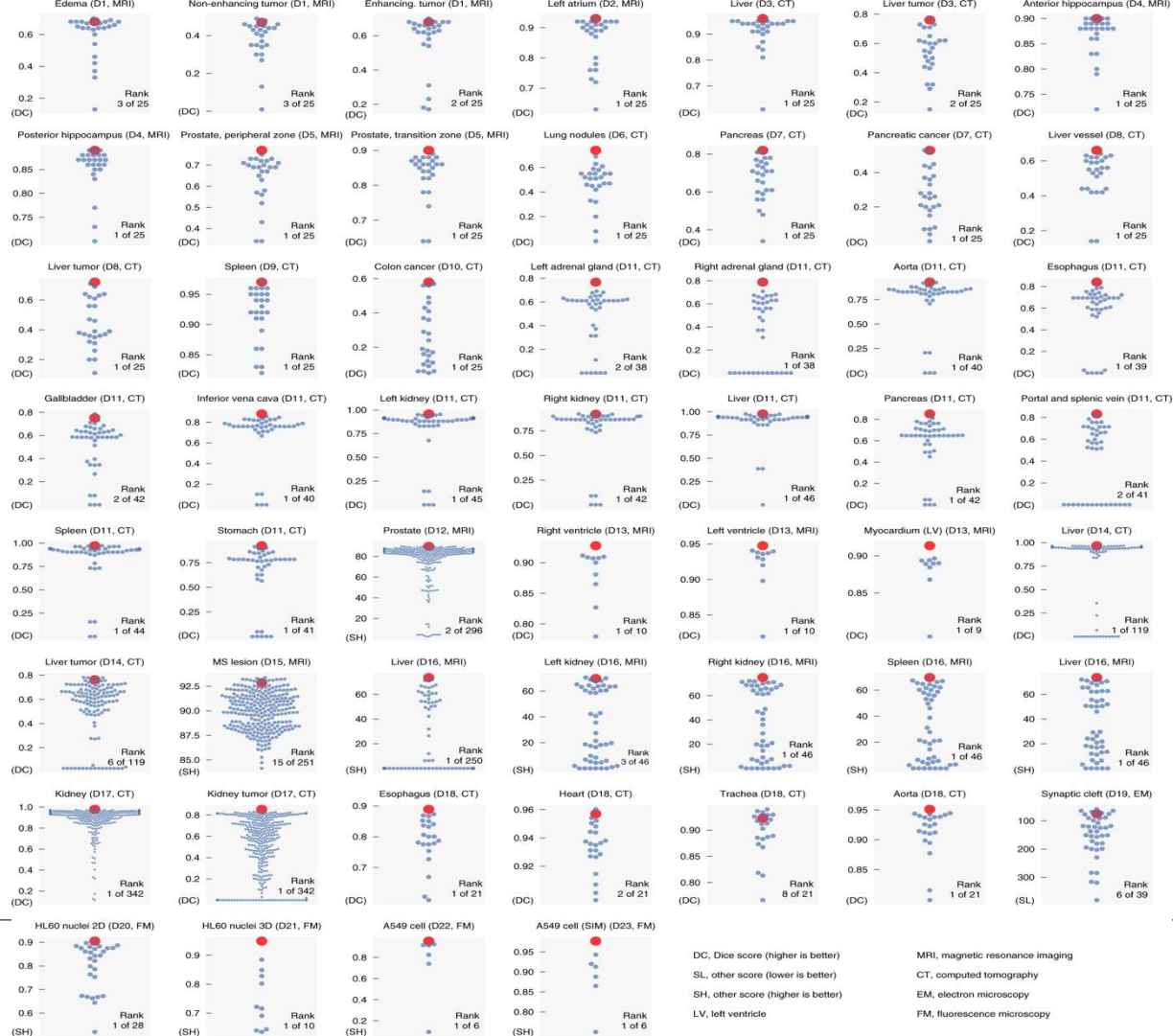
nnU-Net “no new net”

Learning rate	–	Poly learning rate schedule (initial, 0.01)
Loss function	–	Dice and cross-entropy
Architecture template	–	Encoder–decoder with skip-connection ('U-Net-like') and instance normalization, leaky ReLU, deep supervision (topology-adapted in inferred parameters)
Optimizer	–	SGD with Nesterov momentum ($\mu = 0.99$)
Data augmentation	–	Rotations, scaling, Gaussian noise, Gaussian blur, brightness, contrast, simulation of low resolution, gamma correction and mirroring
Training procedure	–	1,000 epochs $\times$ 250 minibatches, foreground oversampling



nnU-Net

First ranked method in
33/53 problems!



Thank you

c.m.scannell@tue.nl